

Concert Tracker

Student Workshop 8w

Version 7.1Y

Project Description

You will build **Concert Tracker**, a command-line application that a live-music company uses to manage its shows. It is a real back-office tool: it keeps track of the **venues** where concerts happen, the **artists** who headline them, the **promoters** who put them on, and the **concerts** themselves — and it can answer real business questions about all of it.

This is a bigger build than the daily exercises. Where those each practiced one idea, the workshop pulls the **whole week** together: several related kinds of data, full management of each, real searches, and reports that *calculate* things from the data. Everything stores in a **MySQL database** using the Spring Boot and JPA approach you practiced all week — entities, repositories, and a service layer — so nothing is hard-coded or kept only in memory.

In IntelliJ, use **Spring Initializr** (File → New → Project → Spring Boot) to create a project named `concert-tracker` — Java 17, Spring Boot 4.0.6 — with the **Spring Data JPA** and **MySQL Driver** dependencies. (If IntelliJ doesn't show Spring Boot, generate it at start.spring.io and open the folder.) **Share it to GitHub from inside IntelliJ** and commit and push often as you work.

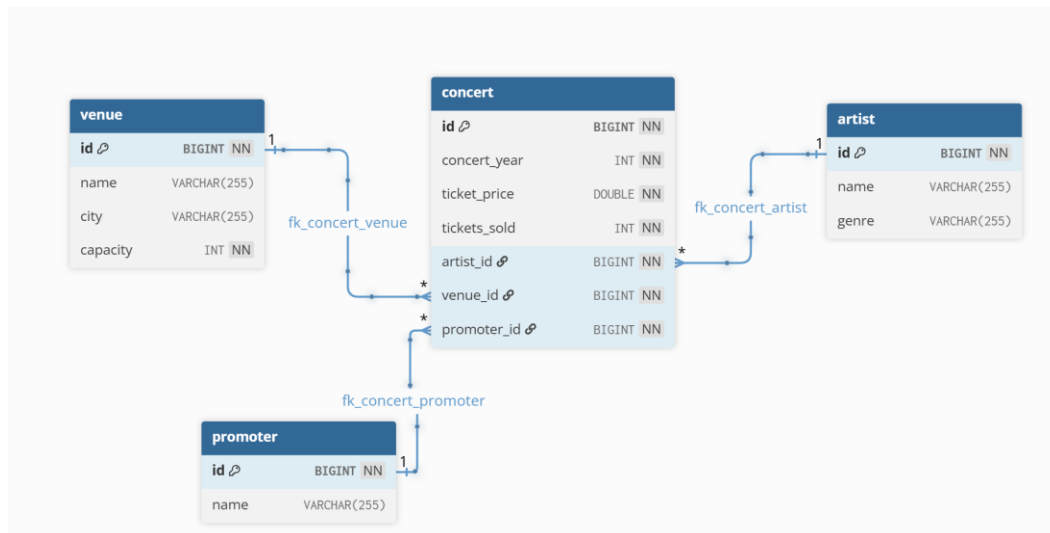
Read **all** of this brief before you write any code, and plan your data and your menu in your notebook first. The work is organized into **four phases** — build them in order; each one builds on the last.

The Data

Concert Tracker is built around **four** kinds of information that connect to one another.

- A **venue** is a place that hosts concerts. Track its **name**, the **city** it's in, and its **capacity** (how many people it can hold).
- An **artist** is a musical act that can headline a concert. Track its **name** and its **genre** (rock, pop, jazz, hip-hop, ...).
- A **promoter** is the person or company that puts on a concert. Track its **name**.
- A **concert** is a single show. Track the **year** it takes place, its **ticket price**, and how many **tickets have been sold**. Every concert also has exactly **one** **headlining artist**, **one venue** that hosts it, and **one promoter** that runs it.

The relationships are the heart of this project. A **concert** points to one artist, one venue, and one promoter — but each **artist** can headline many concerts over time, each **venue** can host many, and each **promoter** can run many. Every concert must have all three; a concert with no venue (or no artist, or no promoter) should never exist.



You'll create the database yourself, one time, and let the app build the tables from your entity classes (the `ddl-auto` setting you used all week). You'll also seed starter data — that's part of Phase 1 below.

How This Workshop Is Organized

Build Concert Tracker in **four phases**, in order. Each one stands on the one before it, so get a phase working and **commit and push** before you move on.

- **Phase 1 — Set Up & Model the Data:** the entities, the relationships, the repositories and service, seed data, and a first listing.
- **Phase 2 — Manage the Catalog:** full add/find/update/delete for all four kinds of data, plus the business rules.
- **Phase 3 — Search & Filter:** the ways a user can slice the concert list.
- **Phase 4 — Reports:** calculations across the data — totals, counts, and averages.

There's an optional **Bonus** section at the end if you finish early.

The Screens

The app talks to the user through a **main menu** that branches into a separate screen for each area — not one long list of thirty-odd options. You have built nested screens like this in earlier capstones and workshops, so use the same idea here: the main menu repeats until the user **quits**, and each area screen repeats until the user goes **back**. On every screen, give clear headings; an unknown choice or a missing id should print a friendly message and stay on that screen — never crash.

- **Main Menu** — the home screen; keeps running until the user quits.
 - 1) Concerts
 - 2) Search concerts
 - 3) Artists
 - 4) Venues
 - 5) Promoters
 - 6) Reports
 - 0) Quit
- **Concerts screen** — list all concerts · view one by id · add a concert · update its price · update its tickets sold · delete · back.
- **Search concerts screen** — by year · by artist · by venue · by city · by maximum price · by price range · advanced (a maximum price **and** an earliest year together) · back.
- **Artists screen** — list · add · find by genre · find by name · update genre · delete · back.
- **Venues screen** — list · add · find by city · find by name · find by minimum capacity · update capacity · delete · back.
- **Promoters screen** — list · add · find by name · delete · back.
- **Reports screen** — revenue per venue · busiest venue & artist · average price by year · capacity report · back.

The phases below walk you through building toward these screens, one area at a time.

Phase 1 | Set Up and Model the Data

The goal of this phase is a working foundation: your data model, saved in MySQL, with starter data you can see on screen.

Model the four entities

- Create **Venue**, **Artist**, **Promoter**, and **Concert** with the fields described in *The Data*.
- Wire up the three relationships: each concert has one artist, one venue, and one promoter, and each of those can appear in many concerts. Make all three **required**, so a concert can never exist without them.

Set up the data layer

- Give each of the four entities its own **repository**.
- Put a **service** between your menu and the repositories. From here on, the menu talks *only* to the service, and the service talks to the repositories — the menu never touches a repository directly.
- Create the database once in MySQL, then let the app build the tables from your entities.

Seed and verify

- When the database is **empty**, seed some starter data: several venues, several artists, a few promoters, and a dozen or so concerts that link them together. Create the venues, artists, and promoters first — a concert needs all three before it can exist. Check that the database is empty before seeding, so a second run doesn't pile on duplicates.
- Add a menu option to **list all concerts**, each shown together with its artist's name and its venue's name (never bare ids).

Done with Phase 1 when the app runs, the tables are created from your entities, and *list all concerts* shows your seeded shows with their artist and venue. Commit and push.

Phase 2 | Manage the Catalog

Now make every kind of data fully manageable from the menu, and enforce the rules that keep it sensible. Whenever the user chooses something from a list (an artist, a venue), show that list **with ids** first so they can pick. Confirm each change on screen, and if the user names an id that doesn't exist, print a clear message and return to the menu — never crash.

Manage concerts

- **Add a concert** — let the user choose its artist, its venue, and its promoter from what already exists, then enter the year, ticket price, and tickets sold.
- **View one concert** by its id, showing its artist, venue, promoter, year, price, and tickets sold.
- **Update** a concert's ticket price, and (separately) its tickets-sold count.
- **Delete** a concert.

Manage venues

- **Add, list, update a venue's capacity, and delete** venues.
- **Find venues by city** — every venue in a city the user names.
- **Find venues by name** — venues whose name contains the user's text.
- **Find venues by minimum capacity** — venues that can hold at least a number the user enters.

Manage artists

- **Add, list, update an artist's genre, and delete** artists.
- **Find artists by genre** — every artist of a genre the user names.
- **Find artists by name** — artists whose name contains the user's text.

Manage promoters

- **Add, list, and delete** promoters.
- **Find promoters by name** — promoters whose name contains the user's text.

The rules

- A concert can **never sell more tickets than its venue's capacity**. If adding a concert or updating its tickets-sold would exceed the venue's capacity, refuse it with a clear message and leave the data unchanged.
- Every concert must keep its **artist, venue, and promoter** — there is no concert missing any of the three.
- Reject obviously bad input (a negative ticket price or a negative tickets-sold count) with a clear message instead of saving it.

Done with Phase 2 when you can add, find, update, and delete all four kinds of data through the menu, and the capacity rule holds. Commit and push.

Phase 3 | Search and Filter Concerts

Let users slice the concert list. Some of these reach across a relationship or apply more than one condition at once — exactly the cases a `findBy...` method name can't handle cleanly, so you'll write **custom queries** with the `@Query` / JPQL approach from this week. Print every result cleanly (each concert with its artist and venue), and when nothing matches, say so instead of printing an empty list.

- **By year** — every concert in a given year.
- **By artist** — every concert whose headliner's name contains the user's text.
- **By venue** — every concert at a named venue.
- **By city** — every concert held in a given city. (This one reaches *through* the venue to its city.)
- **By maximum price** — every concert at or below a price the user enters.
- **By price range** — every concert whose price falls between a minimum and a maximum.
- **By a combination** — a maximum ticket price **and** an earliest year, together, in a single search.

Done with Phase 3 when every search works, and the multi-condition ones genuinely apply both conditions. Commit and push.

Phase 4 | Reports

This is the part that makes Concert Tracker more than a list — it **calculates** things. Each report aggregates across many rows (totals, counts, averages), which you'll do with **aggregate queries** (sum, count, average) written in JPQL. Lay the numbers out so they're easy to read — money as currency, capacity as a percentage — and if a report has no data to show, say so cleanly.

- **Revenue per venue** — for each venue, the total money its concerts have brought in (ticket price times tickets sold, added up).
- **Busiest venue and busiest artist** — the venue that hosts the most concerts, and the artist who headlines the most.
- **Average ticket price by year** — for each year that has concerts, the average ticket price that year.
- **Capacity report** — for each concert, how full it is: tickets sold compared with the hosting venue's capacity, shown as a percentage, and flagged when the show is sold out.

Done with Phase 4 when all four reports print correct numbers from your data. Commit and push — that's the core project complete.

Bonus (Optional)

Once all four phases work, pick any of these to push further:

- **A dashboard** — one menu option that prints a summary of the whole system at once: how many concerts, artists, and venues there are; total revenue; and the busiest venue.
- **Sorting** — list concerts sorted by ticket price (cheapest first), or by year.
- **Nearly sold out** — a report of concerts at or above 90% of their venue's capacity but not yet sold out.
- **Real dates** — store a full concert **date** rather than just a year, and add a search for concerts between two dates.
- **A fourth relationship of your own** — for example, a **Tour** that groups several of an artist's concerts, with a way to view every concert on a tour.

What Makes a Good Workshop Project?

- **You should:**
 - Have a clean, intuitive console interface — a short **main menu** that branches into a screen per area (Concerts, Search, Artists, Venues, Promoters, Reports), each returning to the menu, rather than one long list of options.
 - Complete all four phases — managing venues, artists, promoters, and concerts; the searches; the reports; and the business rules — with every concert linked to its artist, venue, and promoter.
- **You should adhere to best practices such as:**
 - A Spring Boot project that follows the Maven folder structure.
 - Appropriate Java packages (such as `models`, `data`, and `service`) and classes, with the menu calling the service and the service calling the repositories — never the menu reaching a repository directly.
 - Meaningful class names in PascalCase and good variable names in camelCase.
 - Code that is properly formatted, easy to read, and commented where it helps.
- **Make sure that:**
 - Your code is free of errors and that it compiles and runs.
 - Your queries do real work — use custom JPQL where a method name isn't enough, especially for the city search, the multi-condition searches, and the reports.
- **The GitHub repo for your project should be public**, and include a `README.md` that describes your project and lists what it can do.